Αναγνώριση Προτύπων - Εργαστήριο 1

Αναγνώριση φωνής με Κρυφά Μαρκοβιανά Μοντέλα και Αναδρομικά Νευρωνικά Δίκτυα

Κόρδας Νικόλαος (03121032)
Κριθαρίδης Κωνσταντίνος (03121045)

November 13, 2025

Βήμα 9

Πρώτα από όλα κάνουμε clone τα δεδομένα σε μορφή .wav από το repository της εργασίας στον υπολογιστή μας.

Κατόπιν, υλοποιούμε τη συνάρτηση *parse_free_digits* προκειμένου να διαβάσουμε σωστά τα αρχεία με τη βιβλιοθήκη librosa.

```
def parse_free_digits(directory):
    # Parse relevant dataset info
    files = glob(os.path.join(directory, "*.wav"))
    fnames = [f.split("/")[1].split(".")[0].split("_") for f in files]
    ids = [f[2] for f in fnames]
    y = [int(f[0]) for f in fnames]
    speakers = [f[1] for f in fnames]
    _, Fs = librosa.core.load(files[0], sr=None)

    def read_wav(f):
        wav, _ = librosa.core.load(f, sr=None)

        return wav

    # Read all wavs
    wavs = [read_wav(f) for f in files]

    # Print dataset info
    print("Total wavs: {}. Fs = {} Hz".format(len(wavs), Fs))

    return wavs, Fs, ids, y, speakers
```

Το αποτέλεσμα της κλήσης της συνάρτησης είναι:

Total wavs: 3000. Fs = 8000 Hz

Στην συνέχεια βασισμένοι στην υπολοποίηση του βοηθητικού κώδικα ορίζουμε τη δικιά μας συνάρτηση *split_free_digits* την οποία παρουσιάζουμε παρακάτω:

```
def extract_features(wavs, n_mfcc=6, Fs=8000):
    # Extract MFCCs for all wavs
    window = 30 * Fs // 1000
    step = window // 2
    frames = [
        librosa.feature.mfcc(
            y=wav, sr=Fs, n_fft=window, hop_length=window - step, n_mfcc=n_mfcc
        ).T
        for wav in tqdm(wavs, desc="Extracting mfcc features...")
```

```
    ]

    print("Feature extraction completed with {} mfccs per frame".format(n_mfcc)
    )

    return frames


def split_free_digits(frames, ids, speakers, labels):
    print("Splitting in train test split using the default dataset split")
    # Split to train-test
    X_train, y_train, spk_train = [], [], []
    X_test, y_test, spk_test = [], [], []
    test_indices = ["0", "1", "2", "3", "4"]

    for idx, frame, label, spk in zip(ids, frames, labels, speakers):
        if str(idx) in test_indices:
            X_test.append(frame)
            y_test.append(label)
            spk_test.append(spk)
        else:
            X_train.append(frame)
            y_train.append(label)
            spk_train.append(spk)

    return X_train, X_test, y_train, y_test, spk_train, spk_test
```

Τρέχουμε τον κώδικα για να επιβεβαιώσουμε την ορθή λειτουργία του:

```
└─ python3 lab.py
Total wavs: 3000. Fs = 8000 Hz
Splitting in train test split using the default dataset split
Train dataset's size is 2400
Train dataset has 60 samples of digit 0
Train dataset has 60 samples of digit 0
Train dataset has 60 samples of digit 1
Train dataset has 60 samples of digit 1
Train dataset has 60 samples of digit 2
Train dataset has 60 samples of digit 2
Train dataset has 60 samples of digit 3
Train dataset has 60 samples of digit 3
Train dataset has 60 samples of digit 4
Train dataset has 60 samples of digit 4
Train dataset has 60 samples of digit 5
Train dataset has 60 samples of digit 5
Train dataset has 60 samples of digit 6
Train dataset has 60 samples of digit 6
Train dataset has 60 samples of digit 7
Train dataset has 60 samples of digit 7
Train dataset has 60 samples of digit 8
Train dataset has 60 samples of digit 8
Train dataset has 60 samples of digit 9
Train dataset has 60 samples of digit 9
```

Βλέπουμε ότι το split που έχουμε κάνει έχει μέγεθος 2400, δηλαδή είναι πράγματι το 80% του 3000. Επιπλέον, είναι ισορροπημένο, αφού υπάρχει ίδιος αριθμός δειγμάτων ήχου σε κάθε κατηγορία - ψηφίο (60). Επομένως ως μετρική του *accuracy* θα είναι αρκετό για να αξιολογήσουμε την επίδοση των ταξινομητών μας (όπως είδαμε και στην προπαρασκευή, στην περίπτωση του ισορροπημένου dataset το F1-score ακολουθεί το ίδιο trend με το accuracy).

Αφού επιβεβαιώσαμε την ορθή λειτουργία της ρουτίνας μας αφαιρούμε - μετατρέπουμε σε σχόλιο τον κώδικα για το debugging.

Βήμα 10

Στο βήμα αυτό υλοποιούμε αναγνώριση ψηφίων από ομιλία με GMM-HMM. Η λειτουργία του μοντέλου φαίνεται διαγραμματικά παρακάτω:

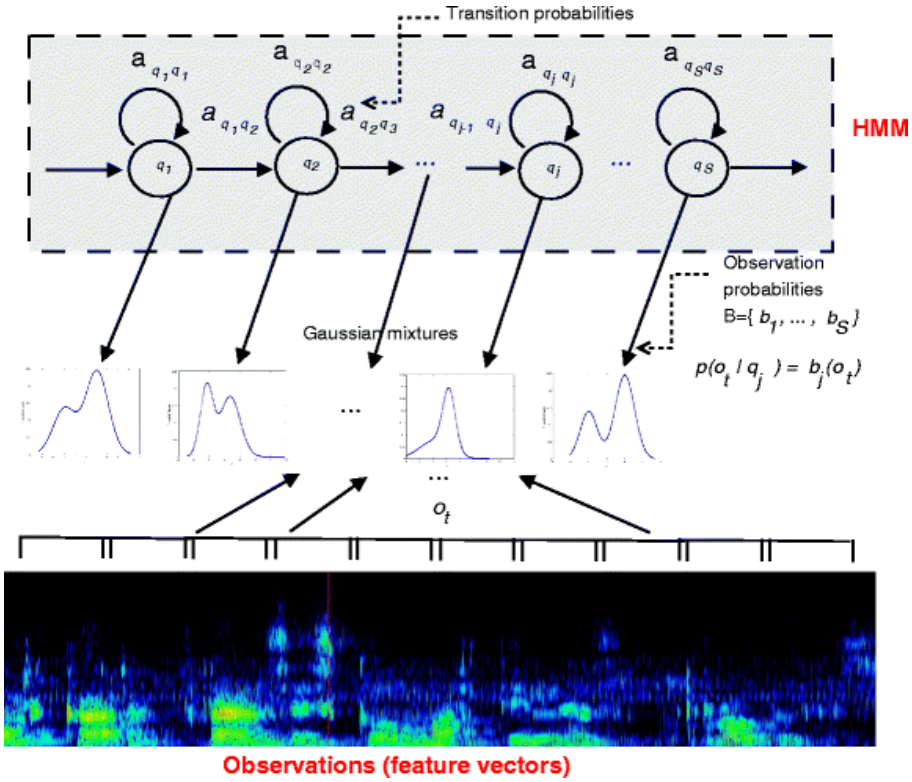


Figure 1: Image by [**?**]

Δημιουργούμε ένα HMM ανά ψηφίο, οπότε η έξοδος του i-οστού HMM θα είναι η πιθανότητα το σήμα να αντιστοιχεί στο i-οστό ψηφίο. Οι παρατηρήσεις σε κάθε state του HMM είναι ένα διαφορετικό mixture of gaussians. Αφού εκπαιδευτεί το GMM από μόνο του σε όλα τα δεδομένα, θα εκπαιδευτεί από κοινού το GMM-HMM μοντέλο και το μείγμα κατανομών ανά state θα γίνει fit στην παρατήρηση που συμβαίνει σε αυτό το state. Ιδανικά, θα θέλαμε εξ'αρχής να μην εκπαιδεύουμε το GMM σε όλα τα δεδομένα, αλλά μόνο σε αυτά που αντιστοιχούν στο χρονικό διάστημα που περιγράφεται από το state. Κάτι τέτοιο, όμως, είναι δυστυχώς δύσκολο να γίνει, αφού δεν είναι προκαθορισμένο για κάθε ψηφίο το πότε θα γίνει η μετάβαση από το ένα state στο επόμενο, οπότε θα αρκεστούμε στην διαφοροποίηση μεταξύ των GMMs που κάνει στη συνέχεια η από κοινού εκπαίδευση. Τέλος, σημειώνουμε ότι με το μείγμα γκαουσιανών μοντελοποιούμε την κατανομή των MFCCs ανά ψηφίο και state, αφού περιμένουμε πως η κάθε τέτοια κατανομή δεν θα είναι unimodal στον χώρο των MFCCs λόγω των διαφορών στην εκφώνηση του ίδιου ψηφίου μεταξύ διαφορετικών ομιλητών.

Η συνάρτηση train φαίνεται παρακάτω:

```
def train_hmms(train_dic, labels, n_states, n_mixtures):
    hmms = {}  # create one hmm for each digit

    for dig in labels:
        X, _, _, _ = train_dic[dig]
        # TODO: YOUR CODE HERE
        emission_model = None
        if gmm: emission_model = initialize_and_fit_gmm_distributions(X,
    n_states=n_states, n_mixtures=n_mixtures)
        else: emission_model = initialize_and_fit_normal_distributions(X,
    n_states=n_states)
        hmms[dig] = train_single_hmm(X, emission_model, dig, n_states)
    return hmms
```

Την `initialize_and_fit_normal_distributions` καθώς είναι υποπερίπτωση του GMM για

n_mixtures = 1.

Η αρχική εκπαίδευση του GMM γίνεται ως εξής:

```
def initialize_and_fit_gmm_distributions(X, n_states, n_mixtures):
    # TODO: YOUR CODE HERE
    dists = []
    all_data = np.concatenate(X, dtype=np.float32)
    D = all_data.shape[1]

    global_mean = np.mean(all_data, axis=0)
    global_std = np.std(all_data, axis=0)

    variance_floor = 1e-6
    base_variance = np.maximum(global_std**2, variance_floor).astype(np.float32
    )

    for _ in range(n_states):
        distributions = []
        for _ in range(n_mixtures):
            means = global_mean + 0.1 * np.random.randn(D).astype(np.float32)

            covs = base_variance.copy()

            distributions.append(
                Normal(means=means, covs=covs, covariance_type=covariance_type)
            )

        gmm_model = GeneralMixtureModel(
            distributions,
            verbose=False,
            max_iter=n_iters,
            random_state=42,
            tol=tol
        ).fit(all_data)
        dists.append(gmm_model)

    return dists
```

Δηλαδή αρχικοποιούμε κάθε γκαουσιανή σε multivariate gaussian με μέσο ειλημμένο από D-διάστατη τυπική κανονική κατανομή, όπου D η διάσταση των δεδομένων, εδώ 13 για τα MFCCs, και με συνδιασπορά τον D-διάστατο ταυτοτικό πίνακα. Επίσης περιορίζουμε κατά την εκπαίδευση τον πίνακα συνδιασποράς να είναι διαγώνιος, ώστε να μειώσουμε τις παραμέτρους και τον χρόνο εκπαίδευσης αλλά και να έχουμε πιο απλές αναπαραστάσεις για τα δεδομένα μας.

Αντίστοιχα, το HMM το εκπαιδεύουμε ως εξής:

```
def train_single_hmm(X, emission_model, digit, n_states):
    A = initialize_transition_matrix(n_states)
    start_probs = initialize_starting_probabilities(n_states)
    end_probs = initialize_end_probabilities(n_states)
    data = [x.astype(np.float32) for x in X]

    model = DenseHMM(
        distributions=emission_model,
        edges=A,
        starts=start_probs,
        ends=end_probs,
        verbose=True,
        max_iter=n_iters,
        tol=tol,
        random_state=42
    ).fit(data)
```

```
    return model
```

Για πίνακα μετάβασης χρησιμοποιήσαμε τον $A = [\alpha_{i,j}]_{1 \leq i,j \leq n}$, όπου $n =$ n_states το πλήθος καταστάσεων στα HMMs, με:

$$\alpha_{i,j} = \begin{cases} 1, & \text{if } j = i = n \\ 0.8 & \text{if } j = i \neq n \\ 0.2 & \text{if } j = i + 1 \\ 0 & \text{αλλιώς} \end{cases}$$

Οι τιμές 0.8 για την ανακύκλωση και 0.2 για την μετάβαση επιλέχθηκαν επειδή το μοντέλο πιο συχνά θα επιλέγει να μείνει στην ίδια κατάσταση, παρά να μεταβεί στην επόμενη.

```
def initialize_transition_matrix(n_states):
    # TODO: YOUR CODE HERE
    # Make sure the dtype is np.float32
    A = np.full((n_states, n_states), 1e-6 ,dtype=np.float32)
    for i in range(n_states):
        if i+1 < n_states:
            A[i][i] = np.float32(0.8)
            A[i][i+1] = np.float32(0.2)
        else:
            A[i][i] = np.float32(1.0)
    return A
```

Τέλος, αρχικοποιήσαμε τις πιθανότητες εκκίνησης ως 1 στην αρχική κατάσταση και 0 σε όλες τις άλλες, ενώ τις πιθανότητες τερματισμού ως 1 στην τελική κατάσταση και 0 σε όλες τις άλλες.

```
def initialize_starting_probabilities(n_states):
    # TODO: YOUR CODE HERE
    # Make sure the dtype is np.float32
    pi = np.zeros(n_states, dtype=np.float32)
    pi[0] = np.float32(1.0)
    return pi


def initialize_end_probabilities(n_states):
    # TODO: YOUR CODE HERE
    # Make sure the dtype is np.float32
    pi_end = np.zeros(n_states, dtype=np.float32)
    pi_end[-1] = np.float32(1.0)
    return pi_end
```

Βήμα 11

Βήμα 12

Βήμα 13

Βήμα 14

1

Η υλοποίηση του βήματος αυτού βασίζεται στη δομή του βοηθητικού κώδικα για το εργαστήριο και συγκεκριμένα στο αρχείο *lstm.py*. Έχουν συμπληρωθεί τα τμήματα σημασμένα ως TODO και επιπλέον έχουν τροποποιηθεί λειτουργικότητες και προστεθεί επιπλεόν συναρτήσεις ώστε να καλυφθούν τα υποερωτήματα του βήματος.

2

Για την κατασκευή του LSTM δικτύου αξιοποιήσαμε τη βιβλιοθήκη PyTorch. Ο κώδικας φαίνεται παρακάτω:

```
def parse_free_digits(directory):
    # Parse relevant dataset info
    files = glob(os.path.join(directory, "*.wav"))
    fnames = [f.split("/")[1].split(".")[0].split("_") for f in files]
    ids = [f[2] for f in fnames]
    y = [int(f[0]) for f in fnames]
    speakers = [f[1] for f in fnames]
    _, Fs = librosa.core.load(files[0], sr=None)

    def read_wav(f):
        wav, _ = librosa.core.load(f, sr=None)

        return wav

    # Read all wavs
    wavs = [read_wav(f) for f in files]

    # Print dataset info
    print("Total wavs: {}. Fs = {} Hz".format(len(wavs), Fs))

    return wavs, Fs, ids, y, speakers
```

Κατά την αρχικοποίηση του δικτύου ελέγχεται κατά πόσο η αρχιτεκτονική του είναι αμφίδρομη ή όχι. Εφόσον είναι, αφενός τίθεται σε True η αντίστοιχη παράμετρος και αφετέρου φροντίζουμε ώστε να διπλασιαστεί το μέγεθος του hidden state του δικτύου.

Στη συνέχεια, επισημαίνουμε τα είδη των layers που θα χρησιμοποιηθούν στην υλοποίηση του δικτύου. Το κυρίως layer είναι το nn.LSTM, το οποίο ρυθμίζεται με τις παραμέτρους του με τρόπο τέτοιο ώστε να καλύψει τις απαιτήσεις της εκφώνησης στα πιθανά ερωτήματα που θα προκύψουν. Το dropout layer δεν χρησιμοποιείται πάντα. Αυτό το καθορίζουμε με την παράμετρο dropout του nn.Dropout layer. Εάν αυτή είναι 0 προφανώς αυτή η τεχνική regularization δεν χρησιμοποιείται. Επιπλέον, αξίζει να αναφέρουμε ότι έχουμε φροντίσει το Dropout να μην εφαρμοστεί στο τελευταίο layer. Το nn.Linear layer χρησιμοποιείται ώστε να πάρει τα output του LSTM (για αυτό έχει και μέγεθος όσο του hidden state) και να τα συνδυάσει προκειμένου να πάρει την τελική απόφαση για το classification του ψηφίου.

Η συνάρτηση forward είναι αυτή που καθορίζει το πώς γίνεται το inference του μοντέλου. Δηλαδή, κατ' επέκταση, τη διάταξη των δομικών layer που ορίσαμε και εξηγήσαμε παραπάνω. Ο πρώτος διαχωρισμός που κάνουμε έγκειται στο αν χρησιμοποιούμε την τεχνική packing στο input (χρειάζεται για το ερώτημα 8 - Bonus). Με την τεχνική του padding πετυχαίνουμε 2 πράγματα. Αρχικά, σε dataset με δεδομένα ακολουθίες διαφορετικών μηκών, είναι σκόπινο να ενώσουμε τις μικρότερες από αυτές ώστε να τις επεξεργαζόμαστε ταυτόχρονα και να εξοικονομούμε χρόνο κατά την εκπαίδευση. Επιπρόσθετα, αποφεύγουμε το zero-padding και την επεξεργασία αυτής της παραμορφωμένης εκδοχής των ακολουθιών εισόδου. Αυτή η τεχνική είναι ιδιαίτερα χρήσιμη σε δίκτυα στα οποία το μέγεθος εισόδου είναι αυστηρά καθορισμένο, όπως οι Transformers. Ο τρόπος που επιλέγουμε να εκτελέσουμε το packing είναι αρχικά να ταξινομήσουμε τις ακολουθίες με τα φωνήματα κατά φθίνον μήκος έτσι ώστε να τις συνδυάσουμε αποτελεσματικότερα. Αυτός ο συνδυασμός πραγματοποιοείται από την συνάρτηση *packed_padded_sequence()* της βιβλιοθήκης της Pytorch (βλέπουμε ότι το enforced_sorted είναι ορισμένο σε True).

Τέλος, όπως βλέπει κανείς εδώ:

```
    def __init__(self, patience, mode="min", base=None, base_model=None):
```

```
        self.best = base
        self.patience = patience
        self.patience_left = patience
        self.mode = mode
```

```
    def stop(self, value: float, model) -> bool:
        # TODO: YOUR CODE HERE
        # Decrease patience if the metric hs not improved
        if self.has_improved(value):
            self.best = value
            self.patience_left = self.patience  # reset patience
            self.model = model
            self.pickle = torch.save(model.state_dict(), checkpoint_filename)
        else:
            self.patience_left -= int(use_early_stopping)

        # Stop when patience reaches zero
        return self.patience_left <= 0

    def has_improved(self, value: float) -> bool:
```

Έτσι εξασφαλίζουμε ότι το dropout δε θα μπει στο τελευταίο layer.

3

Σε αυτό το βήμα καλούμαστε να εκπαιδεύσουμε το μοντέλο. Έτσι, αρχικά, θα εξηγήσουμε τη διαδικασία της εκπαίδευσής του.

```

        return padded

    def __getitem__(self, item):
        return self.feats[item], self.labels[item], self.lengths[item]

    def __len__(self):
        return len(self.feats)


class BasicLSTM(nn.Module):
    def __init__(
        self,
        input_dim,
        rnn_size,
        output_dim,
        num_layers,
        bidirectional=False,
        dropout=dropout,
    ):
        super(BasicLSTM, self).__init__()
        self.bidirectional = bidirectional
        self.feature_size = rnn_size * 2 if self.bidirectional else rnn_size

        # TODO: YOUR CODE HERE
        # ---------------- Insert your code here ---------------- #
        # Initialize the LSTM, Dropout, Output layers
        self.lstm = nn.LSTM(
```

Στο trainning loop παίρνουμε τα δεδομένα του dataloader batch by batch, υπολογίζουμε το loss και κάνουμε το ανάλογο βήμα στον optimizer.

Ο optimizer που χρησιμοποιούμε είναι ο AdamW. Αξιοποιεί όρους ορμής και έχει μοντέλο για να μεταβάλει τους όρους του learning rate και της ταχύτητας κατάλληλα ώστε να επιτύχει την καλύτερη σύγλιση. Προφανώς εκτυπώνουμε το loss σε κάθε εποχή όπως μας ζητείται στην εκφώνηση.

4

Στην συνάρτηση train:

```
        for num_batch, batch in enumerate(dataloader):
            features, labels, lengths = batch

            # TODO: YOUR CODE HERE
            features = features.to(device).float()
            labels = labels.to(device)
            lengths = lengths.to(device)

            # Run forward pass
            logits = model(features, lengths)
            # calculate loss
            loss = criterion(logits, labels)
            running_loss += loss.item()
            # Predict
            outputs = torch.argmax(logits, dim=1) # Calculate the argmax of logits
            y_pred = torch.cat((y_pred, outputs.cpu()))
            y_true = torch.cat((y_true, labels.cpu()))
            num_batches += 1
    valid_loss = running_loss / num_batches
    return valid_loss, y_pred, y_true


def train(models, optimizers, train_dataloader, val_dataloader, criterion):
    # TODO: YOUR CODE HERE
    metrics = {
        "training_loss": {},
        "validation_loss": {},
        "validation_accuracy": {},
    }

    confusion_matrices = {}

    best_model_name, best_val_acc = None, -1

    for model_name, model, optimizer in zip(models.keys(), models.values(), optimizers.values()):
        model.to(device)

        early_stopping = EarlyStopping(patience, mode="min", base_model=None)

        print(f"Training in {device}")

        metrics["training_loss"][model_name] = []
        metrics["validation_loss"][model_name] = []
        metrics["validation_accuracy"][model_name] = []
        for epoch in range(epochs):
            training_loss = training_loop(model, train_dataloader, optimizer, criterion)
            valid_loss, y_pred, y_true = evaluation_loop(model, val_dataloader, criterion)
```

Παίρνουμε τις μετρικές που μας ενδιαφέρουν, δηλαδή το trainning και validation loss καθώς και το validation accuracy. Η master train είναι αυτή που εκπαιδεύει όλα τα μοντέλα που μας ενδιαφέρουν.

5

Καλούμαστε να προσθέσουμε L2 Regularization στο μοντέλο μας, καθώς και dropout.

Το L2 regularization δεν είναι κάτι το οποίο επηρεάζει δομικά το μοντέλο μας άμεσα. Είναι ένας όρος ποινικοποίησης του μέτρου των βαρών που προστίθεται στο loss function. Έτσι, μειώνουμε την πολυπλοκότητα του μοντέλου και έχουμε ως αποτέλεσμα την αποφυγή του overfitting. Αυτό είναι σημαντικό και αναμένεται να παίξει ρόλο, ειδικά σε ένα απλό dataset όπως είναι το δικό μας.

- epochs: 750
- learning rate: $2 * 10^{-}5$
- dropout: 0.4